

CODEX CHAT

Create water level model dashboard

> create a mathematical model dashboard of this water level data and analyze it using python script. Calculate the first derivative including forward difference, central difference, and backward difference. Calculate also the second derivative and find maximum dh/dt . Report the time at maximum and its value in meters/hour. Use cubic mathematical model. Use Levenberg-Marquardt fitting. Calculate fitting statistics. Create two residuals: Residuals vs. time and Residuals vs. fitted water level,

>

> then answer the questions:

>

> - Are the residuals centered around zero?

> - Do they gradually increase/decrease?

> - Are there long groups/runs of positive or negative residuals?

> - Does the spread become larger as the water level increases?

> - Is the largest residual greater than the logger's ± 1 cm accuracy? By how much?

>

> Next calculate the area under the area under the fitted curve. Report Area, Units, Error estimate from `'quad'`. Use meters and hours. Lastly create an html dashboard:

>

> `### Main graph`

>

> - Raw reservoir level vs. time

>

> `### Tab 1 – Derivatives`

>

> - First derivative

> - Second derivative

> - Maximum dh/dt marked

```
>
> ### Tab 2 – Fitted Curve
>
> - &#x20;Raw data&#x20;
> - &#x20;Fitted curve&#x20;
> - &#x20;Model equation&#x20;
>
> ### Residuals
>
> - &#x20;Residual vs. time&#x20;
> - &#x20;Zero line&#x20;
> - &#x20;Your interpretation&#x20;
>
> ### Statistics
>
> Show:
>
> - &#x20;Fitted parameters&#x20;
> - &#x20;Standard errors&#x20;
> - &#x20;t-statistics&#x20;
> - &#x20;p-values&#x20;
> - &#x20;SSE&#x20;
> - &#x20;SST&#x20;
> - &#x20; $R^2$ &#x20;
> - &#x20;Standard error of estimate&#x20;
> - &#x20;Degrees of freedom&#x20;
>
> ### Tab 3 – Area
>
```

> - Fitted curve
> - Shaded area underneath
> - Area value
> - Units
> - Trapezoidal cross-check

<details><summary>2 previous messages</summary>

> I'll locate the water-level dataset, build the cubic Levenberg–Marquardt analysis, and generate a self-contained HTML dashboard plus a reproducible Python script.

>

> <details><summary>Ran 2 commands</summary>

>

> - Ran `rg --files`

> - Ran `Get-Content -Raw lab00\_check\_sandagon.py; Get-Content -Raw tempCodeRunnerFile.python`

>

> </details>

</details>

I can do this, but no water-level dataset is present in the workspace. Please upload or paste the data (CSV/Excel/table) with at least:

- time (hours or timestamps)
- water level (meters)

Once available, I'll generate the Python analysis script and HTML dashboard with all requested derivatives, cubic LM fit, residual diagnostics, statistics, and area calculation.

> User context

>

> Attachments:

> - Data 01.xlsx: `~/Downloads/Data 01.xlsx`

<details><summary>6 previous messages</summary>

> I found the spreadsheet location. I'll read it, preserve the source file unchanged, and place the analysis script and dashboard in the project workspace.

>

> <details><summary>Ran 2 commands</summary>

>

> - Ran `Get-Item -LiteralPath 'C:\Users\stewa\Downloads\Data 01.xlsx' | Select-Object FullName,Length,LastWriteTime; @`

> import pandas as pd

> path = r"C:\Users\stewa\Downloads\Data 01.xlsx"

> xls = pd.ExcelFile(path)

> print(xls.sheet_names)

> for sheet in xls.sheet_names:

> df = pd.read_excel(path, sheet_name=sheet)

> print(f"\n--- {sheet} ---")

> print(df.head(12).to_string(index=False))

> print(df.shape)

> '@ | python -`

> - Ran `@`

> try:

> import openpyxl

> print('openpyxl', openpyxl.__version__)

> except Exception as e: print(type(e).__name__, e)

> try:

> import scipy

```

> print('scipy', scipy.__version__)
> except Exception as e: print(type(e).__name__, e)
> try:
> import plotly
> print('plotly', plotly.__version__)
> except Exception as e: print(type(e).__name__, e)
> '@ | python -`
>
> </details>
>
> The analysis libraries are available, but the spreadsheet-reader package is not. I'm using the XLSX's
native XML contents instead, so the deliverable remains runnable without adding dependencies.
>
> <details><summary>Created a file, ran 2 commands</summary>
>
> - Ran `@'
> from zipfile import ZipFile
> from xml.etree import ElementTree as ET
> path=r'C:\Users\stewa\Downloads\Data 01.xlsx'
>
NS={'m':'http://schemas.openxmlformats.org/spreadsheetml/2006/main','r':'http://schemas.openxmlfo
rmats.org/officeDocument/2006/relationships'}
> with ZipFile(path) as z:
> print('\n'.join(z.namelist()))
> wb=ET.fromstring(z.read('xl/workbook.xml'))
>
print([(s.attrib.get('name'),s.attrib.get('{http://schemas.openxmlformats.org/officeDocument/2006/rela
tionships}id')) for s in wb.find('m:sheets',NS)])
> ss=[]
> if 'xl/sharedStrings.xml' in z.namelist():

```

```

> root=ET.fromstring(z.read('xl/sharedStrings.xml'))

> ss=[".join(t.text or "" for t in
si.iter('{http://schemas.openxmlformats.org/spreadsheetml/2006/main}t')) for si in root]

> print('shared strings',ss[:50])

> for n in [x for x in z.namelist() if x.startswith('xl/worksheets/sheet')]:

> print('\n',n)

> root=ET.fromstring(z.read(n))

> for row in root.findall('.//m:sheetData/m:row',NS)[:20]:

> vals=[]

> for c in row.findall('m:c',NS):

> v=c.find('m:v',NS); value="" if v is None else v.text

> if c.attrib.get('t')=='s' and value: value=ss[int(value)]

> vals.append((c.attrib.get('r'),value))

> print(vals)

> '@ | python -`

> - Wrote `./water_level_dashboard.py` (+244 -0)

> - Ran `python water_level_dashboard.py --input 'C:\Users\stewa\Downloads\Data 01.xlsx' --output
water_level_dashboard.html`

>

> </details>

>

> The dashboard has been generated. One strong result is already clear: the cubic is a poor
representation of part of this record—the largest residual is 2.7825 m, far beyond the 1 cm logger
accuracy—so the residual diagnostics are meaningful rather than merely cosmetic.

>

> <details><summary>Edited a file, ran 2 commands</summary>

>

> - Ran `python -m py_compile water_level_dashboard.py; Get-Content -Raw
water_level_dashboard.json; Get-Item water_level_dashboard.html,water_level_dashboard.py | Select-
Object Name,Length,LastWriteTime`

> - Wrote `./water_level_dashboard.py` (+8 -3)

```

```
> - Ran `python -m py_compile water_level_dashboard.py; python water_level_dashboard.py --input  
'C:\Users\stewa\Downloads\Data 01.xlsx' --output water_level_dashboard.html`
```

```
>
```

```
> </details>
```

```
</details>
```

Created the analysis and self-contained dashboard:

- [Python script](/C:/Users/stewa/python-practice/water_level_dashboard.py)
- [HTML dashboard](/C:/Users/stewa/python-practice/water_level_dashboard.html)
- [Machine-readable results](/C:/Users/stewa/python-practice/water_level_dashboard.json)

Cubic LM model, with time in elapsed hours:

$h(t) = -0.00010595068t^3 + 0.009606124t^2 - 0.091243028t + 14.150575$

Key results:

- Maximum fitted dh/dt : **0.19907 m/h**
- Time of maximum: **2026-07-22 06:13 PHT** (30.222 h elapsed)
- Area under fitted curve: **1261.19984 m·h**
- `quad` error estimate: **$\pm 1.40 \times 10^{-11}$ m·h**
- Trapezoidal cross-check: **1261.19850 m·h**

Fit statistics:

Metric	Value
SSE	322.47584 m ²

| SST | 2034.07794 m² |

| R² | 0.841463 |

| Standard error of estimate | 1.06559 m |

| Degrees of freedom | 284 |

Residual assessment:

- Centered around zero: **Yes** (mean $\approx$ 0 m).
- Gradual increase/decrease: **No** material overall linear trend.
- Long positive/negative runs: **Yes** — longest run is **81** readings.
- Spread larger at higher levels: **Not clearly** (high/low spread ratio 1.03).
- Largest residual: **2.7825 m (278.25 cm)**, which exceeds 1 cm logger accuracy by **2.7725 m (277.25 cm)**.

The dashboard includes the requested raw-level plot, all three first-difference methods, second derivative, fitted curve, both residual plots, parameter statistics, and shaded area plot.

> Add more design in the html dashboard and put the statistics along with the fitted curve & residuals in tab 2. Don't change anything else, just the design and moving statistic tab.

<details><summary>2 previous messages</summary>

> I'll keep the calculations untouched, enhance the dashboard styling, and move the full statistics section into Tab 2 alongside the fitted curve and residuals.

>

> <details><summary>Edited a file, ran a command</summary>

>

> - Wrote `./water\_level\_dashboard.py` (+3 -4)

> - Ran `python -m py\_compile water\_level\_dashboard.py; python water\_level\_dashboard.py --input 'C:\Users\stewa\Downloads\Data 01.xlsx' --output water\_level\_dashboard.html; rg "id='statistics'|>Statistics<" water\_level\_dashboard.html`

>

> </details>

</details>

Updated and regenerated the dashboard.

- Enhanced visual design: layered header, refined cards, tab styling, visual hierarchy, table styling, and responsive layout.
- Moved all fitting statistics into **Tab 2 – Fitted curve & residuals**.
- Removed the separate Statistics tab.
- Kept all calculations and reported values unchanged.

Updated files:

- [Dashboard script](/C:/Users/stewa/python-practice/water_level_dashboard.py)
- [Dashboard HTML](/C:/Users/stewa/python-practice/water_level_dashboard.html)

> add effects on tab transitions, don't change anything else

<details><summary>2 previous messages</summary>

> I'll only enhance the existing tab-transition animation and regenerate the HTML; the analysis, layout, and content will remain unchanged.

>

> <details><summary>Edited a file, ran a command</summary>

>

> - Wrote `./water\_level\_dashboard.py` (+1 -1)

> - Ran `python -m py\_compile water\_level\_dashboard.py; python water\_level\_dashboard.py --input 'C:\Users\stewa\Downloads\Data 01.xlsx' --output water\_level\_dashboard.html`

>

> </details>

</details>

Added smooth fade, rise, and subtle scale effects when switching tabs, with reduced-motion support.
Regenerated the dashboard without changing its data or calculations.

[Updated dashboard](/C:/Users/stewa/python-practice/water_level_dashboard.html)

> did you update the python script too?

Yes. The transition effects were added to [water_level_dashboard.py](/C:/Users/stewa/python-practice/water_level_dashboard.py), then it regenerated
[water_level_dashboard.html](/C:/Users/stewa/python-practice/water_level_dashboard.html).

I checked this by analyzing the output data if it shows realistic and relevant value.